ISYE 7406 HW2

Introduction

The purpose of this assignment is to analyze the “fat” dataset to better understand a variety of statistical modeling techniques and their roles in building predictive models. These models are also compared by their testing error on the testing set, as well as utilizing a Monte Carlo Cross Validation to compare average testing errors across a hundred replications.

Exploratory Data Analysis

The dataset consists of 252 rows (observations) and 18 columns (variables). The first column, brozek, is the response variable, and is a measure of body fat percentage using Brozek’s equation. The remaining 17 variables are possible predictors in the formation of several models later on. Following assignment guidelines, 25 of the 252 observations (10%) are explicitly selected to form the testing data–fat1test–while the remainder (90%) serve as training data–fat1train.

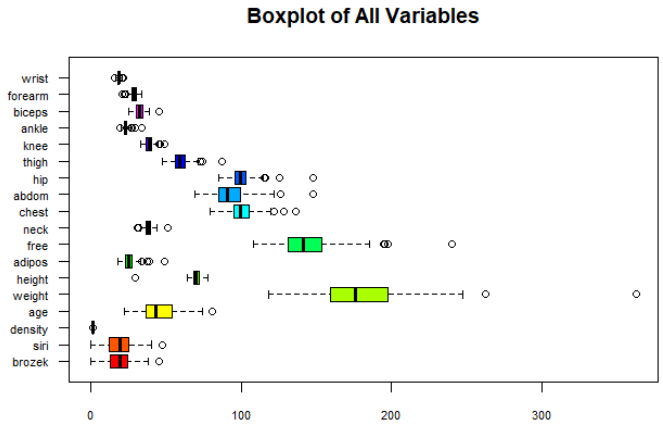

Figure 1 (Left): Boxplots of every variable in the fat dataset

Figure 1 showcases a group of boxplots, with the response variable at the bottom. Immediately observed are the visible outliers among many of the variables.

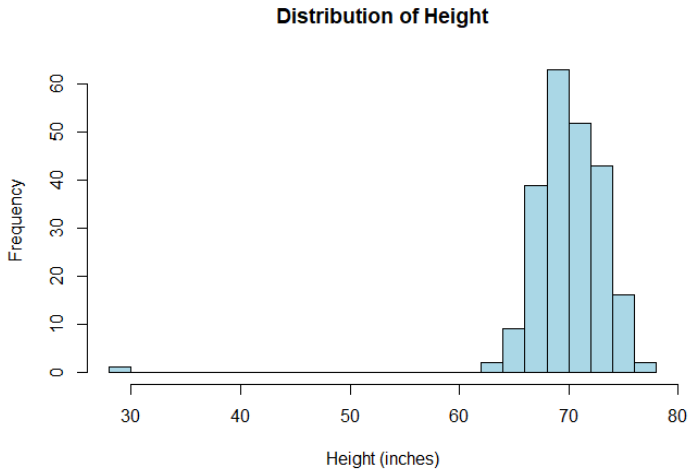

Figure 2 (Left): Histogram of height distribution in the fat dataset, in

Figure 2 illustrates a relatively normal distribution of height. The mean and median are relatively close at 70.16 and 70.00 inches, respectively. There is an extreme range difference of 48.25, which can be attributed to an extreme outlier with a height of 29.50 inches. Although this specific analysis and modeling techniques will include every observation, it is highly reasonable for anyone to exclude this observation in order to avoid skewing analysis.

Table 1 (below): Variance inflation factor of the first model.

```
> vif(model1)
      siri    density        age     weight     height     adipos       free
neck
 77.729464  42.051164   2.282681 118.165700   2.222689  18.210994  67.291096
4.465353
     chest      abdom        hip      thigh       knee      ankle     biceps
forearm
 11.349572  22.256555  16.010538   8.355124   4.951599   1.962964   3.834843
2.633857
     wrist
  3.636941
```

Table 1 showcases the correlation between individual predictors when compared to all other predictors. The higher the number, the more correlated and troublesome, as over 10 is widely considered to be a threshold for eliminating or combining variables. There is an alarming amount of multicollinearity in this dataset. For instance, weight has a VIF of 118, which can be attributed to factors such as height, adiposity, and the circumference of the anatomical variables (almost every predictor). While the inclusion of weight would certainly help to achieve a much higher R^2 value, this effect is meaningless as there are other predictors that may achieve similar effects to weight, and is therefore a valid reason to exclude weight for modeling purposes. This is an example of “less is more”, and there are modeling techniques later on in this analysis to address this problem.

Methods

Seven models are built using fat1train:

1. Linear Regression with all predictors
2. Linear Regression with the best subset of k = 5 predictors
    a. The five predictors are siri, density, thigh, knee, and wrist
3. Linear Regression with variables (stepwise) selected using AIC
4. Ridge Regression
5. LASSO
6. Principal component regression
    a. Optimal PCs is 17 in PCR

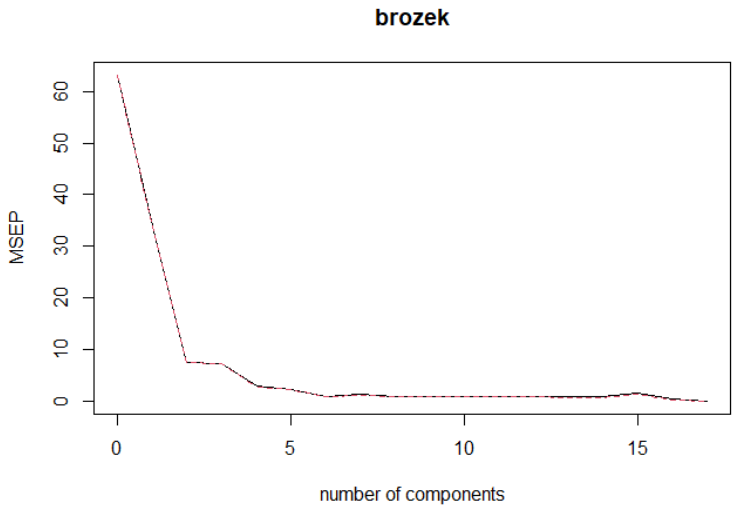

Figure 3 (Right): Validation plot showing least MSEP at 17

7. Partial least squares
    a. Optimal PCs is 17 in PLS

Each model will undergo predictions using the testing data, and will be compared by their testing error when comparing the predicted values to the actual values.

Lastly, the Monte Carlo Cross-Validation algorithm repeats this step 100 times to obtain an average testing error value among all 7 models as there are quite few observations and relatively many predictors.

Results

LinReg	**LinRegk=5**	Stepwise
0.008755981	**0.002786218**	0.008955971

Ridge	LASSO	PCR	PLS
0.008859234	0.003158102	0.008755981	0.007922117

Table 2 (Left): Testing error for all models in a single run

From table 2, the lowest testing error indicates best model performance, and this is the case for model 2–Linear Regression using k = 5. This method selected the best 5 predictors to be used in a linear regression model, and demonstrates that it can significantly downplay the effects of overfitting, as is the case for model 1 (all predictors).

Model 3 (Stepwise) uses AIC for feature selection, which prioritizes simplicity of model and the likelihood of the model. Model 3 has the highest testing error in this single run.

Model 4 and 5 (Ridge and LASSO) are regularization techniques. While LASSO can eliminate features by setting coefficients to zero, Ridge will shrink its importance but not eliminate it. LASSO has a much lower testing error than Ridge possibly because of its ability to eliminate redundant variables.

Model 6 (PCR) performs the same as model 1 because the optimal principal components is 17, which is exactly the amount of predictors in the dataset. This model suggests that using all components is optimal for accuracy, as opposed to its need to use fewer components to avoid overfitting.

Model 7 (PLS) shares the same number of PCs, and while PCR focuses only on the predictors' variance, PLS focuses on maximizing the covariance between predictors and response variable (multicollinearity). In the end, model 7 had a lower testing error than model 6.

```
MCCV Mean
LinReg      LinRegk=5   Stepwise
0.04378772 0.03598465 0.04331040
Ridge       LASSO       PCR         PLS
0.04410632 0.05337448 0.04378772 0.04289242
MCCV Variance
LinReg       LinRegk=5    Stepwise
0.003000114 0.002357988 0.003239202
Ridge        LASSO        PCR          PLS
0.003040646 0.005043047 0.003000114 0.003184926
```

Table 3 (above): Mean and standard deviation of the MCCV testing errors

After 100 runs through MCCV, it appears that model 5 now has the highest testing error among all models, while model 2 remains the model with the best performance on the testing data. The same can be said for variance results.

Findings

A simpler model using the best subset of predictors performs best on the testing data, and one must be wary of highly correlated predictors for modeling. With a relatively small sample size, the Monte Carlo Cross-Validation technique enhances the effectiveness of testing models on unseen data, as the performance of a model can change when comparing a single simulation to a hundred.

Appendix

```
# Setup

fat <- read.table(file = "fat.csv", sep=',', header=TRUE)
n = dim(fat)[1]
n1 = round(n/10)
flag = c(1, 21, 22, 57, 70, 88, 91, 94, 121, 127, 149, 151, 159, 162,
         164, 177, 179, 194, 206, 214, 215, 221, 240, 241, 243);
fat1train = fat[-flag,]; fat1test = fat[flag,]

# EDA
summary(fat1train)
boxplot(fat1train,
        main = "Boxplot of All Variables",
        col = rainbow(ncol(fat1train)),
        las = 1,
        cex.axis = 0.7,
        horizontal = TRUE)
hist(fat1train$height,
     main = "Distribution of Height",
     xlab = "Height (inches)",
     col = "lightblue",
     border = "black",
     breaks = 30)
hist(fat1train$weight,
     main = "Distribution of Height",
     xlab = "Height (inches)",
     col = "lightblue",
     border = "black",
     breaks = 30)
hist(fat1train$age,
     main = "Distribution of Age",
     xlab = "Years",
     col = "lightblue",
     border = "black",
     breaks = 30)
boxplot(fat1train$abdom,
        main = "Boxplot of Abdominal Circumference",
        ylab = "Abdominal Circumference (cm)",
        col = "lightgreen",
        border = "darkgreen",
        notch = TRUE)
cor(fat1train)

# Model 1. Linear Regression
model1 <- lm(brozek ~ ., data=fat1train)
summary(model1)

# VIF for EDA
library(car)
vif(model1)

# Model 1 Testing Error
```

```r
model1_pred <- predict(model1, newdata=fat1test[,-1])
model1_te <- mean((model1_pred - fat1test[,1])^2)
print(model1_te)

# Model 2. Linear Regression using k = 5 predictors variables
library(leaps)
five_subset <- regsubsets(brozek ~., data=fat1train, nvmax=5)
subset_summary <- summary(five_subset)
print(subset_summary$which)
model2 <- lm(brozek ~ siri + density + thigh + knee + wrist, data=fat1train)
summary(model2)

# Model 2 Testing Error
model2_pred <- predict(model2, newdata=fat1test[,-1])
model2_te <- mean((model2_pred - fat1test[,1])^2)
print(model2_te)

# Model 3. Linear regression with variables (stepwise) selected using AIC;
model3 <- step(model1, direction="both")
summary(model3)

# Model 3 Testing Error
model3_pred <- predict(model3, newdata=fat1test[,-1])
model3_te <- mean((model3_pred - fat1test[,1])^2)
print(model3_te)

# Model 4. Ridge Regression
library(MASS)
model4 <- lm.ridge(brozek ~., data=fat1train, lambda = seq(0, 100, 0.001))
plot(model4)
matplot(model4$lambda, t(model4$coef), type = "l", lty = 1,
        xlab = expression(lambda), ylab = expression(hat(beta)))
indexopt <- which.min(model4$GCV)
ridge_coeff <- model4$coef[, indexopt] / model4$scales
intercept <- -sum(ridge_coeff * colMeans(fat1train[, -1])) +
mean(fat1train$brozek)
ridge_coeffs <- c(intercept, ridge_coeff)

# Model 4 Testing Error
model4_pred <- as.matrix(fat1test[,-1]) %*% as.vector(ridge_coeff) + intercept
model4_te <- mean((model4_pred - fat1test[, 1])^2)
print(model4_te)

# Model 5. LASSO
library(lars)
fat_lasso <- lars(as.matrix(fat1train[,-1]), fat1train$brozek, type="lasso",
trace=TRUE)
plot(fat_lasso)
Cp_lasso <- summary(fat_lasso)$Cp
index_lasso <- which.min(Cp_lasso)
lasso_lambda <- fat_lasso$lambda[index_lasso]
coef_lasso <- predict(fat_lasso, s = lasso_lambda, type = "coef", mode =
"lambda")$coef
lasso_intercept <- mean(fat1train$brozek) - sum(coef_lasso *
colMeans(fat1train[, -1]))
lasso_coeffs <- c(lasso_intercept, coef_lasso)
```

```r
# Model 5 Testing Error
model5_pred <- predict(fat_lasso, as.matrix(fat1test[, -1]), s = lasso_lambda,
type = "fit", mode = "lambda")
model5_te <- mean((model5_pred$fit - fat1test[,1])^2)
print(model5_te)

# Model 6. PCR
library(pls)
model6 <- pcr(brozek ~., data=fat1train, scale=TRUE, validation="CV")
summary(model6)
validationplot(model6, val.type = "MSEP")
opt_pcs_pcr <- which.min(model6$validation$adj)

# Model 6 Testing Error
model6_pred <- predict(model6, newdata=fat1test[-1], ncomp=opt_pcs_pcr)
model6_te <- mean((model6_pred - fat1test[,1])^2)
print(model6_te)

# Model 7. Partial Least Squares
model7 <- plsr(brozek ~., data=fat1train, validation="CV")
summary(model7)
opt_pcs_pls <- which.min(model7$validation$adj)

# Model 7 Testing Error
model7_pred <- predict(model7, newdata=fat1test[,-1], ncomp=opt_pcs_pls)
model7_te <- mean((model7_pred - fat1test[,1])^2)
print(model7_te)

# All TE's
model_names <- c("LinReg", "LinRegk=5", "Stepwise", "Ridge", "LASSO", "PCR",
"PLS")
te_vals <- c(model1_te, model2_te, model3_te, model4_te, model5_te, model6_te,
model7_te)
names(te_vals) <- model_names
print(te_vals)

# MCCV
set.seed(123)
B = 100
TEALL = NULL
library(glmnet)
model1_te_mccv <- numeric(B)
for (i in 1:B) {
  flag <- sort(sample(1:n, n1));
  fattraintemp <- fat[-flag,]
  fattesttemp <- fat[flag,]

  # Model 1
  model1 <- lm(brozek ~ ., data=fattraintemp)
  model1_pred <- predict(model1, newdata=fattesttemp[,-1])
  te0 <- mean((model1_pred - fattesttemp[,1])^2)

  # Model 2
  five_subset <- regsubsets(brozek ~ ., data = fattraintemp, nvmax = 5)
  selected_vars <- names(which(summary(five_subset)$which[5,]))
  model2 <- lm(as.formula(paste("brozek ~", paste(selected_vars[-1], collapse =
"+"))), data = fattraintemp)
```

```
  model2_pred <- predict(model2, newdata = fattesttemp)
  te1 <- mean((model2_pred - fattesttemp$brozek)^2)

  # Model 3
  model3 <- step(model1, direction="both")
  model3_pred <- predict(model3, newdata=fattesttemp[,-1])
  te2 <- mean((model3_pred - fattesttemp[,1])^2)

  # Model 4
  model4 <- lm.ridge(brozek ~., data=fattraintemp, lambda = seq(0, 100, 0.001))
  indexopt <- which.min(model4$GCV)
  ridge_coeff <- model4$coef[, indexopt] / model4$scales
  intercept <- -sum(ridge_coeff * colMeans(fattraintemp[,-1])) +
mean(fattraintemp$brozek)
  ridge_coeffs <- c(intercept, ridge_coeff)
  model4_pred <- as.matrix(fattesttemp[,-1]) %*% as.vector(ridge_coeff) +
intercept
  te3 <- mean((model4_pred - fattesttemp[,1])^2)

  # Model 5
  lasso_model <- cv.glmnet(as.matrix(fattraintemp[, -1]), fattraintemp$brozek,
alpha = 1)
  model5_pred <- predict(lasso_model, s = "lambda.min", newx =
as.matrix(fattesttemp[, -1]))
  te4 <- mean((model5_pred - fattesttemp[,1])^2)

  # Model 6
  model6 <- pcr(brozek ~., data=fattraintemp, scale=TRUE, validation="CV")
  opt_pcs_pcr <- which.min(model6$validation$adj)
  model6_pred <- predict(model6, newdata=fattesttemp[-1], ncomp=opt_pcs_pcr)
  te5 <- mean((model6_pred - fattesttemp[,1])^2)

  # Model 7
  model7 <- plsr(brozek ~., data=fattraintemp, validation="CV")
  opt_pcs_pls <- which.min(model7$validation$adj)
  model7_pred <- predict(model7, newdata=fattesttemp[,-1], ncomp=opt_pcs_pls)
  te6 <- mean((model7_pred - fattesttemp[,1])^2)

  TEALL = rbind(TEALL, cbind(te0, te1, te2, te3, te4, te5, te6))
}
apply(TEALL, 2, mean)
apply(TEALL, 2, var)
```